Documentation interfaces

This document contains the description of the interfaces used in the file “main_interfaz.f90”.

Note: the variables that contain an asterisk are **optional** arguments.

`read_clean_line`

- **Description:** Internal subroutine that reads the next non-blank, non-comment line from standard input. It skips empty lines and full-line comments (lines whose first non-blank character is "!" or "#"), so the input file driven by stdin redirection can be annotated. The caller then parses the returned buffer with an internal list-directed read (e.g. `read(buf, *, iostat=ios) value`), which preserves the original type-safe conversion to integers, reals and character strings.
- **Class:** Internal procedure of program `main_interfaz` (contained subroutine).
- **Input: none.**

- **Output:**

Variable	Type	Description
`line`	String	Next clean input line, left-adjusted and with leading blanks stripped
`ios`	Integer	I/O status (0 on success, non-zero on read error or end-of-file)

- **Process:**
  - Reads one line from standard input into `line` using a character-format read `read(*, '(A)', iostat=ios) line`. On read error or end-of-file, `ios` is propagated to the caller and the subroutine returns immediately.
  - Left-adjusts the line with `adjustl` and computes its trimmed length with `len_trim`. Blank lines are skipped (the loop reads the next line).
  - If the first non-blank character is "!" or "#", the line is treated as a full-line comment and skipped. Only full-line comments are stripped: "!" or "#" appearing in the middle of a line are kept verbatim, so file paths containing those characters are preserved. Comments must therefore be placed on their own line.
  - Returns the trimmed, left-adjusted line to the caller in `line` with `ios = 0`.

`read_chemistry_interface`

- **Description:** Subroutine that reads all the chemical input data (species, reactions, initial and external concentrations, mineral zones, etc.) by dispatching to the appropriate format-specific reader.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
`root`	String	Root of input and output files of problem to be solved
`path_pb`	String	Path to input and output files of problem to be solved
`path_DB`	String	Path to chemical databases
`num_tar`	Integer	Expected number of target (domain) waters from the mesh

- **Pre-process:**
  - Sets arbitrary file units for the chemical system, computation options and local chemistry input files.
- **Process**:
  - Selects the input format option (`read_opt`) and reads the **chemical data** accordingly. Currently only the CHEPROO-based format (`read_opt` = 1) is fully implemented; the PHREEQC (`read_opt` = 2) and PFLOTRAN (`read_opt` = 3) branches abort execution with an error stop.

write_conc_comp_tar_wat

- **Description:** Subroutine that writes the aqueous component and variable activity species concentrations of every target water and every external water to a file. A reference water type whose name contains **domain**, **initial**, **resident** (or their Spanish equivalents) is located, and its primary-species basis is used to project the variable-activity-species concentrations of every water onto a common component vector, so all targets yield component vectors of identical dimension. Waters are split into a domain group (entries of tar_wat_indices) and an external group (entries of ext_waters_indices, after filtering out any water whose name matches the reference water type), and each group is written separately.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
`path`	String	Path to the output file
`filename`	String	Output file name

- **Process**:
  - Locates the reference water type by case-insensitive name matching **(domain / initial / resident / dominio / inicial / residente)** and aborts with an error stop if none is found.
  - Classifies waters into two groups: target waters (indices in tar_wat_indices) and external waters (indices in ext_waters_indices), filtering out from the external set any water whose name matches the reference water type (those are conceptually domain waters and were already written in the domain block).
  - Opens the output file (`path//filename`) and writes a header with the number of aqueous components.
  - Writes the list of variable-activity aqueous species of the reference water type, with primary species in rows 1 to `num_aq_prim_species` followed by secondary variable-activity species.
  - Writes the aqueous components as linear combinations of the variable-activity species defined by `comp_mat_aq`, for example `u6 = hco3- + co3-2`.
  - For each target water, projects its variable-activity-species concentrations onto the reference basis and writes two scientific-notation tables under the headings "Aqueous component concentrations in the domain:" and "Aqueous variable activity species concentrations in the domain:" (rows: components/species, columns: target waters).

- For each external water, performs the same projection and writes the corresponding aqueous component and variable-activity species concentration tables under the headings "Aqueous component concentrations of external waters:" and "Aqueous variable activity species concentrations of external waters:", preceded by a header row of right-aligned water names (rows: components/species, columns: external waters).
- Closes `filename`

`get_num_aq_comps_tar_wat`

- **Description:** Function that returns the number of aqueous components in a target water. If no index is provided, the first target water is used.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
`ind_tw*`	Integer	Optional index of a target water

- **Output:**

Variable	Type	Description
`num_aq_comps`	Integer	Number of aqueous components in the target water

- **Process**:
  - Returns the value of `num_aq_prim_species` in the `speciation_alg` object associated to the target water.

interfaz_comps_arch_eq

- **Description:** Subroutine that solves a reactive mixing iteration with **components**, and writes aqueous component and variable activity species concentrations, using **files** as arguments. The integration method is **Euler explicit** and the reaction mixing ratios are **lumped**. It assumes that there are equilibrium reactions only (no kinetics), so the aqueous component concentrations after reactive mixing are identical to those after conservative transport.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
path	String	Path to input and output files.
num_aq_comps	Integer	Nº aqueous components.
file_in	String	File containing concentrations of aqueous components after solving a conservative transport iteration $(\tilde{\mathbf{U}}^{k+1})$.
Delta_t	Real	Time step (Δt^k)
file_out	String	File where aqueous component concentrations $(\mathbf{U}^{k+1})$ and variable activity species concentrations $(\mathbf{C}_v^{k+1})$ after solving a reactive mixing iteration will be written.

- **Pre-process:**
  - Allocate aqueous component concentrations after conservative transport $(\tilde{\mathbf{U}}^{k+1})$
  - Determine, from the first target water, the number of variable activity species n_nc.
  - Allocate the variable activity species concentrations conc_nc with shape (n_nc, num_target_waters).
- **Process**:
  - **Read** $\tilde{\mathbf{U}}^{k+1}$ from file_in. **Note:** The rows must represent components and the columns targets.
  - **Solve** reactive mixing iteration for each target water j:
    - **Loop over number of target waters**
      - Speciate to compute variable activity species concentrations $\mathbf{c}_{v,j}^{k+1}$ from u_tilde, using Newton-Raphson.
      - Check Newton convergence: if speciation does not converge for a target water, the target index and a "No convergence in speciation after reactive mixing iteration. Try reducing the time step" message are printed and execution is aborted with an error stop.

- **Post-process:**
  - **Write** $\mathbf{U}^{k+1}$ in `file_out` (same dimensions as $\widetilde{\mathbf{U}}^{k+1}$)
  - **Write** $\mathbf{C}_v^{k+1}$ in `file_out` (rows: species, columns: targets)
  - **Deallocate** arrays

interfaz_comps_arch_eq_T

- **Description:** Transposed-input variant of **interfaz_comps_arch_eq**. Solves a reactive mixing iteration with components and writes aqueous component and variable activity species concentrations, using files as arguments, when there are equilibrium reactions only (no kinetics). The integration method is Euler explicit and the reaction mixing ratios are lumped. The only difference with respect to **interfaz_comps_arch_eq** is that the input file **file_in** has rows as targets and columns as components (transposed with respect to the canonical layout). It is selected at run time by the driver when **flag_transpose = 1**. Inputs, outputs, pre-process, process and post-process are otherwise identical to **interfaz_comps_arch_eq**, except that **file_in** is read row by row and transposed in memory before the speciation step.
- **Class: Chemistry**

`interfaz_comps_arch_eq_kin`

- **Description:** Subroutine that solves a reactive mixing iteration with **components**, and writes aqueous component concentrations, variable activity species concentrations and kinetic reaction rates, using **files** as arguments. The integration method is **Euler explicit** and the reaction mixing ratios are **lumped**. It assumes that there are both equilibrium and kinetic reactions.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
`path`	String	Path to input and output files.
`num_aq_comps`	Integer	Nº aqueous components.
`file_in`	String	File containing concentrations of aqueous components after solving a conservative transport iteration $(\tilde{\mathbf{U}}^{k+1})$.
`Delta_t`	Real	Time step (Δt^k)
`file_out`	String	File where aqueous component concentrations $(\mathbf{U}^{k+1})$ and variable activity species concentrations $(\mathbf{C}_v^{k+1})$ after solving a reactive mixing iteration will be written.

- **Pre-process:**
  - Allocate aqueous component concentrations after conservative transport $(\tilde{\mathbf{U}}^{k+1})$.
  - Allocate chemical reactions contribution to aqueous component concentrations $(\mathbf{U}_K^{k+1})$.
  - Determine, from the first target water, the number of kinetic reactions (aqueous kinetic `n_aq_kin`, mineral kinetic `n_min_kin`) and the number of variable activity species `n_nc`.
  - Allocate the variable activity species concentrations `conc_nc` with shape (`n_nc`, `num_target_waters`).
  - Allocate the kinetic reaction rates `rk_out` with shape (`n_aq_kin` + `n_min_kin`, `num_target_waters`).
- **Process**:
  - **Read** $\tilde{\mathbf{U}}^{k+1}$ from `file_in`. **Note:** The rows must represent components and the columns targets.
  - **Solve** reactive mixing iteration for each target water j:
    - **Loop over number of target waters**
      - Compute chemical reactions contribution to aqueous component concentrations using **Euler explicit** and **lumping**:

$$\mathbf{u}_{K,j}^{k+1} := \Delta t^k \mathbf{W} \mathbf{S}_{K,v}^T \mathbf{r}_{K,j}^k$$

- Update the total aqueous component concentrations in target water: $\mathbf{u}_j^{k+1} = \widetilde{\mathbf{u}}_j^{k+1} + \mathbf{u}_{K,j}^{k+1}$.
- Speciate to compute variable activity species concentrations $\mathbf{c}_{v,j}^{k+1}$ from components, using Newton-Raphson.
- Check Newton convergence: if speciation does not converge for a target water, the target index and a "No convergence in speciation after reactive mixing iteration. Try reducing the time step" message are printed and execution is aborted with an error stop.
- Capture the kinetic reaction rates `rk_out` for the current target water from the aqueous and mineral kinetic rate arrays of the chemistry object.

- **Post-process:**
  - **Write** $\mathbf{U}^{k+1}$ in `file_out` (same dimensions as $\widetilde{\mathbf{U}}^{k+1}$)
  - **Write** $\mathbf{C}_v^{k+1}$ in `file_out` (rows: species, columns: targets)
  - **Write** kinetic reaction rates `rk_out` in `file_out` (rows: reactions [aqueous (linear+redox) | minerals], columns: targets)
  - **Deallocate** arrays

`interfaz_comps_arch_eq_kin_T`

- **Description:** Transposed-input variant of **`interfaz_comps_arch_eq_kin`**. Solves a reactive mixing iteration with components and writes aqueous component concentrations, variable activity species concentrations and kinetic reaction rates, using files as arguments, when there are both equilibrium and kinetic reactions. The integration method is Euler explicit and the reaction mixing ratios are lumped. The only difference with respect to **`interfaz_comps_arch_eq_kin`** is that the input file **`file_in`** has rows as targets and columns as components (transposed with respect to the canonical layout). It is selected at run time by the driver when **`flag_transpose = 1`**. Inputs, outputs, pre-process, process and post-process are otherwise identical to **`interfaz_comps_arch_eq_kin`**, except that **`file_in`** is read row by row and transposed in memory before the kinetic update and speciation steps.
- **Class: Chemistry**

`interfaz_esp_arch`

- **Description:** Subroutine that solves a reactive mixing iteration with **aqueous species**, and writes aqueous species concentrations and kinetic reaction rates, using **files** as arguments. The integration method is **Euler explicit** and the reaction mixing ratios are **lumped**. It assumes that there are no equilibrium reactions.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
`path`	String	Path to input and output files.
`num_aq_comps`	Integer	Nº aqueous species (= nº aqueous components)
`file_in`	String	File containing concentrations of aqueous species after solving a conservative transport iteration $\left(\tilde{\mathbf{C}}^{k+1}\right)$
`Delta_t`	Real	Time step $\left(\Delta t^{k}\right)$
`file_out`	String	File where aqueous species concentrations $\left(\mathbf{C}^{k+1}\right)$ and kinetic reaction rates after solving a reactive mixing iteration will be written

- **Pre-process:**
  - Allocate aqueous species concentrations after conservative transport $\left(\tilde{\mathbf{C}}^{k+1}\right)$
  - Allocate chemical reactions contribution to aqueous species concentrations $\left(\mathbf{C}_{K}^{k+1}\right)$
  - Determine, from the first target water, the number of kinetic reactions (aqueous kinetic `n_aq_kin`, mineral kinetic `n_min_kin`).
  - Allocate the kinetic reaction rates `rk_out` with shape (`n_aq_kin` + `n_min_kin`, `num_target_waters`).
- **Process**:
  - **Read** $\tilde{\mathbf{C}}^{k+1}$ from `file_in`. **Note:** The rows must represent species and the columns targets.
  - **Solve** reactive mixing iteration for each target water j:
    - **Loop over number of target waters**
      - Compute chemical reactions contribution to aqueous species concentrations using **Euler explicit** and **lumping**:

$$\mathbf{c}_{K,j}^{k+1} := \Delta t^{k} \mathbf{S}_{K,aq}^{T} \mathbf{r}_{K,j}^{k}$$

      - Update the total aqueous species concentrations in target water:

$$\mathbf{c}_{j}^{k+1} = \tilde{\mathbf{c}}_{j}^{k+1} + \mathbf{c}_{K,j}^{k+1}$$

      - Capture the kinetic reaction rates `rk_out` for the current target water from the aqueous and mineral kinetic rate arrays of the chemistry object.
- **Post-process:**

- **Write** $\mathbf{C}^{k+1}$ in `file_out` (rows: species, columns: targets)
- **Write** kinetic reaction rates `rk_out` in `file_out` (rows: reactions [aqueous (linear+redox) | minerals], columns: targets)
- **Deallocate** arrays